

Industry-Standard Project Structure

Folder structure, coding rules, and what each piece is actually for



- React
- Node.js
- Express
- PostgreSQL
- Ollama

1. Why Structure Matters	
2. The Full Project Tree	
3. Client (React) — Folder by Folder	
4. Server (Node) — Folder by Folder	
5. Deep Dive: What is Middleware?	
6. Deep Dive: Routes vs Controllers vs Services	
7. Connecting PostgreSQL and Ollama	
8. General Coding Standards	
9. Golden Rules Checklist	

1. Why Structure Matters

Right now, your project is small enough that you could put everything in one file and it would still work. That's exactly why this matters **now** — bad habits are easy to start small and painful to undo later.

THINK OF IT LIKE A KITCHEN

A messy kitchen where knives, vegetables, and dirty dishes are all on one counter still lets you cook — slowly, with mistakes. A professional kitchen has separate stations: prep, cooking, plating. Everyone knows exactly where to go and what their job is. Folder structure is your kitchen layout — it doesn't change what you're cooking, it changes how fast and how safely you can cook it.

A good structure gives you three things:

- **Predictability** — any developer (including a future you) can guess where something lives without searching the whole project
- **Separation of concerns** — a bug in "how data is displayed" doesn't get tangled with a bug in "how data is fetched"
- **Team scalability** — four people can work on four different folders at once without stepping on each other's code

2. The Full Project Tree

This is a monorepo layout — frontend and backend live in the same repository, in separate top-level folders.

```
project-root/
├── client/                # React frontend
│   ├── public/
│   ├── src/
│   │   ├── assets/       # images, icons, fonts
│   │   ├── components/   # reusable UI pieces
│   │   ├── pages/        # full screens
│   │   ├── hooks/        # custom React hooks
│   │   ├── services/     # API call functions
│   │   ├── context/      # global app state
│   │   ├── utils/        # helper functions
│   │   ├── styles/
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── .env
│   └── package.json
├── server/               # Node backend
│   ├── src/
│   │   ├── config/       # DB + Ollama connection setup
│   │   ├── routes/       # URL → function mapping
│   │   ├── controllers/  # handles request/response
│   │   ├── services/     # business logic
│   │   ├── models/       # DB query definitions
│   │   ├── middleware/   # auth, validation, error handling
│   │   ├── utils/
│   │   ├── db/
│   │   │   ├── migrations/ # schema changes over time
│   │   │   └── seeds/      # sample data
│   │   └── app.js
│   ├── server.js         # entry point
│   ├── .env
│   └── package.json
├── .gitignore
├── docker-compose.yml    # postgres + ollama containers
└── README.md
```

3. Client (React) — Folder by Folder

Folder	What Goes Here	Why It's Separate
components/	Small, reusable UI pieces: Button, Card, Modal, InputField	Build once, use everywhere. Change a button style in one place, it updates the whole app.
pages/	Full screens: HomePage, TermDetailPage, SearchPage	Pages assemble components into a complete screen — they're the "final product," components are the "parts."
hooks/	Custom React hooks, e.g. useFetchTerm, useDebounce	Reusable logic that isn't visual — keeps components focused on display, not logic.
services/	Functions that call your backend API	If your API URL or request format changes, you edit one file — not every component that fetches data.

context/	Global state — e.g. current logged-in user, theme	Avoids "prop drilling" (passing data through 5 layers of components just to reach one that needs it).
utils/	Small helper functions: formatDate, truncateText	Pure logic, no UI — easy to test on its own.

Simple rule for React: if a piece of code draws something on screen, it belongs in `components/` or `pages/`. If it fetches or calculates something, it belongs in `services/`, `hooks/`, or `utils/`. Don't mix the two inside one file.

4. Server (Node) — Folder by Folder

Folder	What Goes Here	Why It's Separate
config/	Database connection setup, Ollama connection setup, environment config	All your "how do I connect to X" logic lives in one place, not scattered across the app.
routes/	Just the URL definitions — which path triggers which controller	Lets anyone see the entire API surface by scanning one folder, without reading logic.
controllers/	Reads the incoming request, calls the right service, sends back a response	Keeps "handling a request" separate from "doing the actual work."
services/	The real business logic — database queries, calculations, calling Ollama	Can be reused by multiple controllers, and tested without needing a fake HTTP request.
models/	Defines how data is structured and queried in the database	One place to see/update your DB structure in code, matching your ER diagram.
middleware/	Code that runs <i>before</i> a request reaches the controller — see Section 5	Shared checks (auth, validation) apply to many routes without repeating code.
db/migrations/	Step-by-step scripts that build/change your database tables over time	Every teammate runs the same migrations — everyone's database ends up identical.
db/seeds/	Scripts that insert sample/test data	Instantly get realistic data to develop against, without typing it in manually.

5. Deep Dive: What is Middleware?

Middleware is code that sits **between** the moment a request arrives and the moment your controller actually handles it. Think of it as a checkpoint the request has to pass through first.

THINK OF IT LIKE AIRPORT SECURITY

Before you reach your flight (the controller), you pass through checkpoints: ID check, baggage scan, boarding pass verification. Each checkpoint either lets you through to the next one, or stops you right there and sends you back. Middleware works exactly like this — each one either calls `next()` to let the request continue, or stops it and sends an error response immediately.

Common Middleware You'll Actually Use

- Auth middleware** — checks if the request has a valid login token before letting it reach a protected route (like the Favorites endpoints)
- Validation middleware** — checks if the request body has all required fields, in the right format, before the controller wastes time processing bad data
- Error-handling middleware** — catches errors thrown anywhere in the app and turns them into a clean, consistent error response instead of crashing the server
- Logging middleware** — records every incoming request (useful for debugging "why did this fail?" later)

What It Looks Like in Code

```
// middleware/auth.middleware.js
function checkAuth(req, res, next) {
  const token = req.headers.authorization;

  if (!token) {
    return res.status(401).json({
      success: false,
      error: { code: "UNAUTHORIZED", message: "Login required." }
    });
  }

  // token looks valid — let the request continue
  next();
}

module.exports = checkAuth;
```

```
// routes/favorite.routes.js
const checkAuth = require("../middleware/auth.middleware");

// checkAuth runs FIRST. Only if it calls next(), the controller runs.
router.post("/favorites", checkAuth, favoriteController.create);
```

Why not just check auth inside the controller? Because you'd have to copy-paste that same check into every single protected route. Middleware means you write the check once, and attach it wherever it's needed.

6. Deep Dive: Routes vs Controllers vs Services

This is the layer split students mix up most often. Here's the flow of a single request:



Layer	Job	Should NOT contain
Route	"This URL + method goes to this controller function"	Any actual logic — just mapping
Middleware	"Should this request even be allowed through?"	Business logic like database queries
Controller	"Read the request, call the service, format the response"	Direct database queries
Service	"Actually do the work" — query the DB, call Ollama, calculate something	Anything about HTTP (no req/res objects here)

THE MISTAKE TO AVOID

Writing a database query directly inside a route file. It'll work today, but the moment you need that same logic in two places, or need to test it, or need to swap databases — you're stuck untangling one giant file. Keep the layers separate from day one.

Example: The Full Flow for "Get One Term"

```
// routes/term.routes.js
router.get("/terms/:id", termController.getById);

// controllers/term.controller.js
async function getById(req, res) {
  const term = await termService.findById(req.params.id);
  if (!term) {
    return res.status(404).json({ success: false, error: { code: "TERM_NOT_FOUND" } });
  }
  res.status(200).json({ success: true, data: term });
}

// services/term.service.js
async function findById(id) {
  const result = await db.query("SELECT * FROM terms WHERE id = $1", [id]);
  return result.rows[0];
}
```

7. Connecting PostgreSQL and Ollama

7.1 PostgreSQL Connection

Your database connection setup lives in `config/db.js` — one file, reused everywhere via the services layer.

```
// config/db.js
const { Pool } = require("pg");

const pool = new Pool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: process.env.DB_PORT,
});

module.exports = pool;
```

7.2 Ollama Connection

Ollama runs as its own local service — your backend talks to it over HTTP, the same way it would talk to any external API.

```
// config/ollama.js
const OLLAMA_URL = process.env.OLLAMA_URL || "http://localhost:11434";
module.exports = { OLLAMA_URL };

// services/ollama.service.js
const { OLLAMA_URL } = require("../config/ollama");

async function generateMeaningSummary(text) {
  const response = await fetch(`${OLLAMA_URL}/api/generate`, {
    method: "POST",
    body: JSON.stringify({ model: "llama3", prompt: text }),
  });
  return response.json();
}
```

Why separate config from service? If Ollama's URL changes (e.g. moves to a different server), you change one line in `config/ollama.js` — nothing else in your app needs to know or care.

8. General Coding Standards

8.1 Naming Conventions

- Files: camelCase or kebab-case — pick one and stay consistent (e.g. `term.controller.js`)
- Variables/functions: camelCase — `getTermById`, not `get_term_by_id`
- React components: PascalCase — `TermCard.jsx`, not `termcard.jsx`
- Constants: UPPER_SNAKE_CASE — `MAX_RETRY_COUNT`

8.2 Environment Variables

- Never hardcode passwords, API keys, or URLs directly in code
- Keep all secrets in `.env` files
- `.env` must be listed in `.gitignore` — never commit it to git
- Provide a `.env.example` file with blank/dummy values so teammates know what variables are needed

8.3 Error Handling

- Every async function that can fail should be wrapped in `try/catch`
- Never let raw error messages (stack traces, DB errors) reach the frontend — send a clean, consistent error response instead
- Use one centralized error-handling middleware rather than repeating error logic in every controller

8.4 Git Habits

- One feature = one branch. Never commit directly to `main`
- Commit messages describe *what changed and why*, not "fixed stuff" or "update"
- Pull before you push — always sync with the latest branch state first
- Small, frequent commits are easier to review than one giant commit at the end of the day

9. Golden Rules Checklist

BEFORE YOU WRITE A SINGLE LINE OF FEATURE CODE, CONFIRM:

- Does this logic belong in a route, controller, service, or middleware — have I placed it correctly?
- Am I about to write a database query directly inside a route or controller? (Stop — move it to a service)
- Is any password, API key, or URL hardcoded anywhere? (Stop — move it to `.env`)
- Does this protected route have the right middleware attached (auth, validation)?
- Am I about to commit directly to `main`? (Stop — create a branch)
- Does my folder/file naming match the conventions the rest of the team is using?